

Understanding Spec-Driven-Development

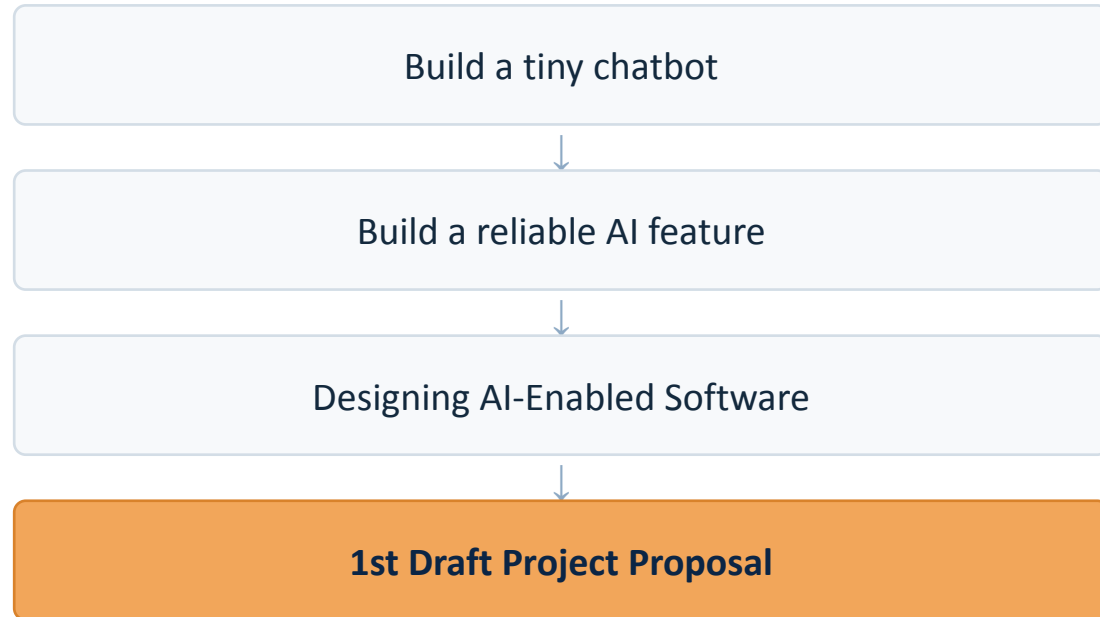
Review

1. What's the difference between an AI demo and an AI-enabled software system?
2. What AI Fit Framework?
3. Why RAG exists?
4. List three things that separate a prototype (prompt + model + UI) from a production AI system.

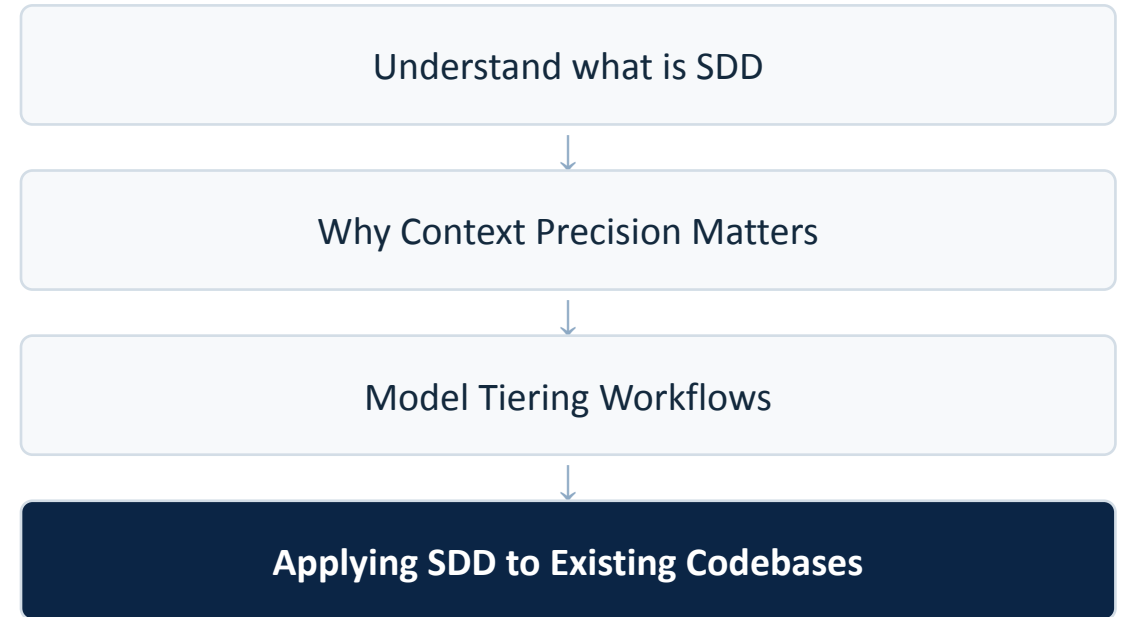
WEEK 4

Where We Are

WEEK 1–3



WEEK 4 – Understanding Spec-Driven-Development (SDD)



Learning Objectives

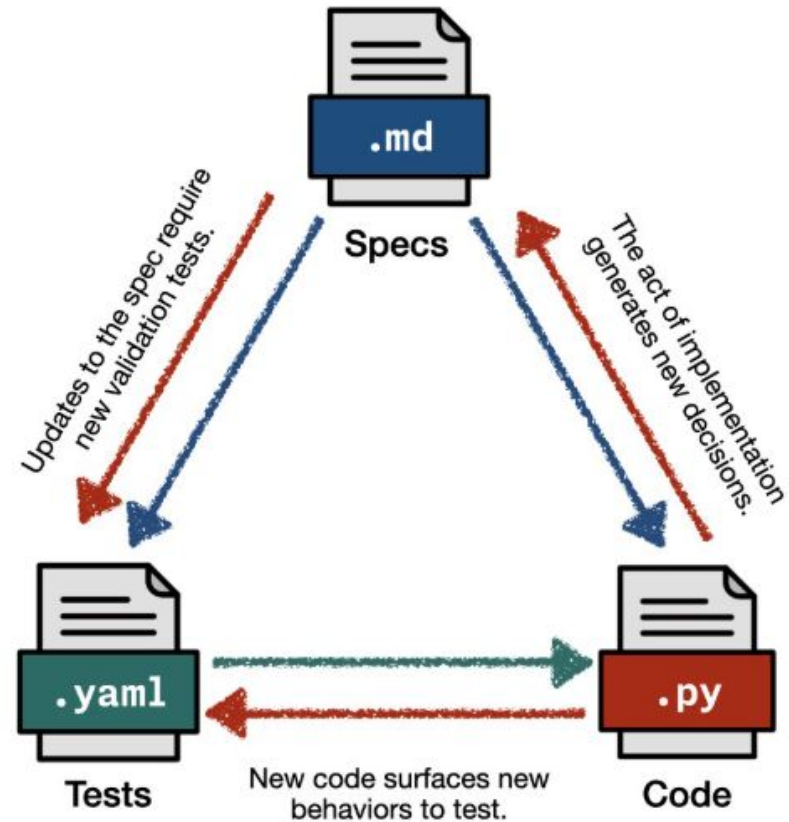
- 1 What Is Spec-Driven Development?
- 2 Why Spec-Driven Development Matters Now
- 3 How SDD Differs
- 4 Elements of a Good Spec
- 5 When to Use a Spec, and When Not To

What Is Spec-Driven Development?

(SDD)

What Is Spec-Driven Development?

Spec-driven development is a methodology that treats specifications as executable contracts from which AI agents derive code, preventing architectural drift through automated enforcement rather than passive documentation.



Why Spec-Driven Development Matters Now

Three forces converged in 2025-2026 that make SDD the workflow I default to when AI-generated code needs to survive in production.

- LLMs generate vulnerable code at rates ranging from **9.8% - 42.1%**
- Compliance requirements now treat specifications as evidence (EU AI Act)
- Distributed architectures demand formal governance

Why Spec-Driven Development Matters Now

The Data-Backed Case: Why AI-Generated Code Needs Specification Gates:

1. A [SonarQube analysis of five LLMs](#) (arXiv, Aug 2025) generating Java code found over 70% of Llama 3.2 90B's detected vulnerabilities rated BLOCKER severity, and roughly two-thirds of GPT-4o's and OpenCoder-8B's rated BLOCKER or CRITICAL. The pattern repeats across the literature.

Why Spec-Driven Development Matters Now (cont.)

The Data-Backed Case: Why AI-Generated Code Needs Specification Gates:

2. [Pearce et al.](#) (IEEE S&P, 2023) found roughly 40% of programs generated in security-sensitive contexts contained vulnerabilities. [Yan et al.](#) (2025) put the range at 9.8% to 42.1% across their benchmarks. A catalog by [Fu et al.](#) (ACM TOSEM, 2025) identified 43 CWEs across three AI code-generation tools. By February 2026, a [large-scale empirical study](#) (arXiv, 2026) had counted more than 110,000 surviving AI-introduced issues in production repositories.

Why Spec-Driven Development Matters Now (cont.)

The Data-Backed Case: Why AI-Generated Code Needs Specification Gates:

2. [Pearce et al.](#) (IEEE S&P, 2023) found roughly 40% of programs generated in security-sensitive contexts contained vulnerabilities. [Yan et al.](#) (2025) put the range at 9.8% to 42.1% across their benchmarks. A catalog by [Fu et al.](#) (ACM TOSEM, 2025) identified 43 CWEs across three AI code-generation tools. By February 2026, a [large-scale empirical study](#) (arXiv, 2026) had counted more than 110,000 surviving AI-introduced issues in production repositories.

How SDD Differs from PRDs, Design Docs, TDD, and BDD

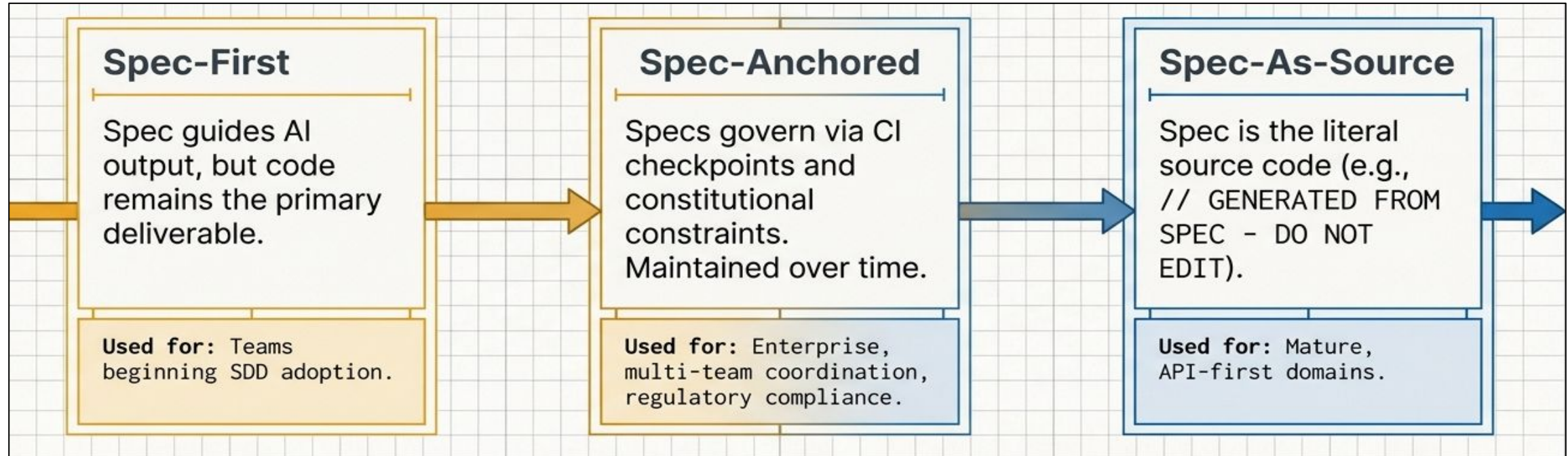
	TDD	BDD	Vibe Coding	SDD
Primary Artifact	Unit tests	Given-When-Then scenarios	Natural language prompts	Executable specifications
Scope	Individual function correctness	Cross-functional behavior	Full app generation	System-wide architectural contracts
Validation	Automated test suites	Human-referenced docs	Manual review (if any)	Build fails on spec divergence
Where Truth Lives	Test suite	Workshop artifacts	Prompt history	Versioned specification

The Six Elements of a Good Spec

A spec for an AI agent needs to answer six questions. Leave any of them open and the agent will answer them for you, in ways you won't like.

1. Outcomes
2. In-scope and explicitly out-of-scope boundaries
3. Constraints and assumptions (Tech stack limit, API limit ...)
4. Prior decisions
5. Task breakdown
6. Verification criteria

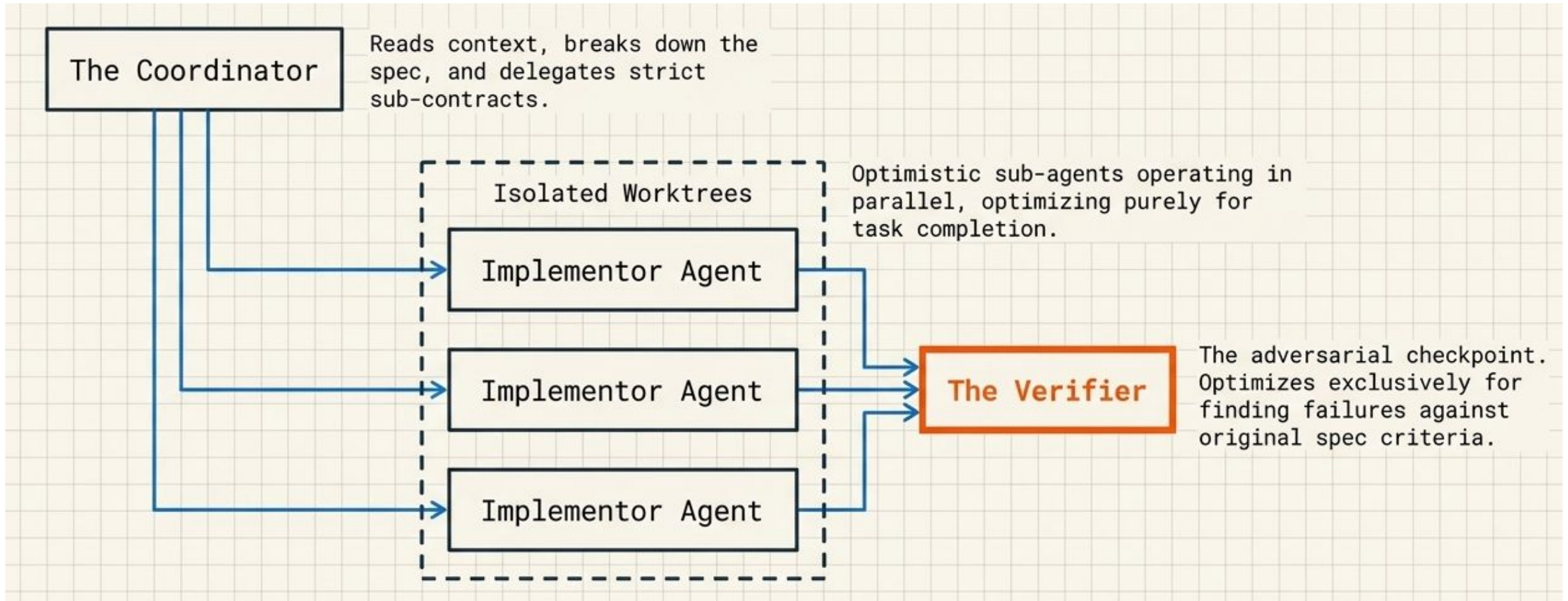
Core SDD Patterns



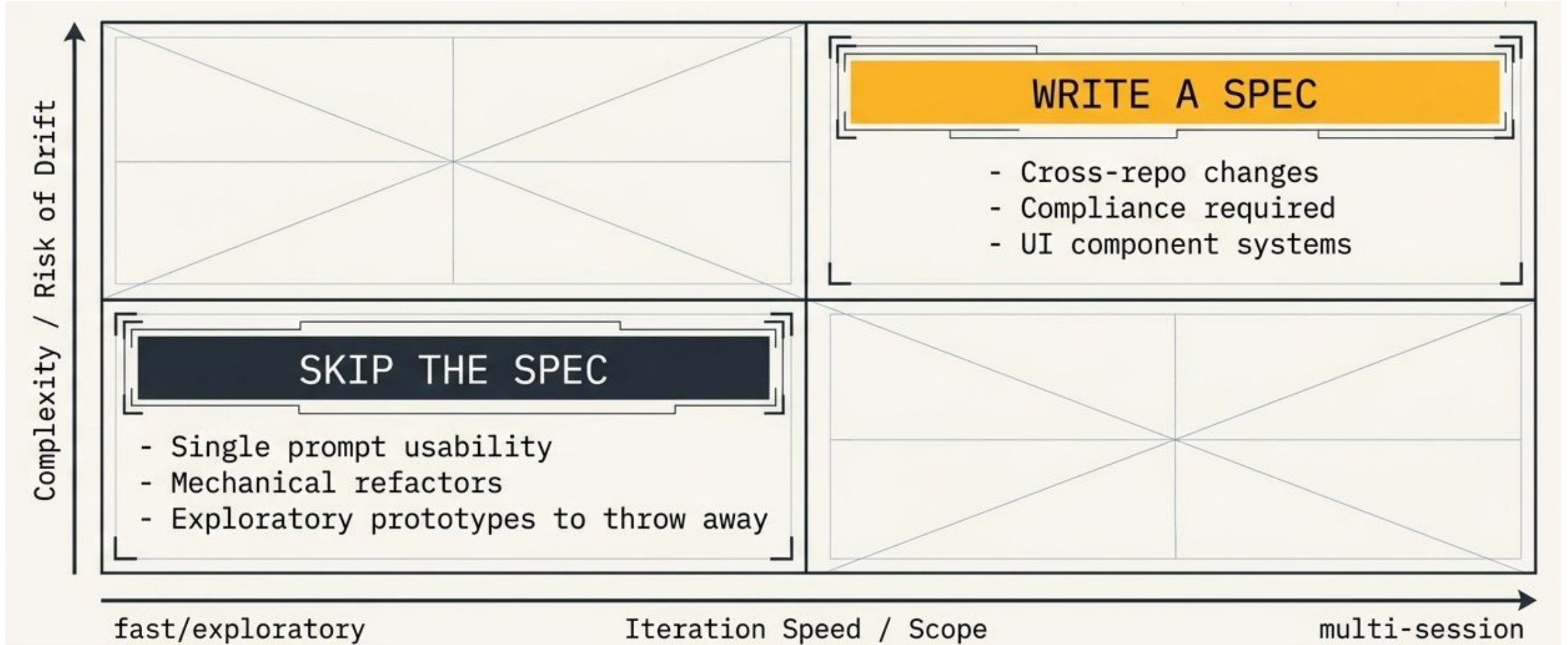
Core SDD Patterns (cont.)

- **Spec-first development** come before code and constrain what AI agents generate, while code remains the primary deliverable.
- **Spec-anchored development** adds governance layers, constitutional constraints, and supervision checkpoints. I reach for this pattern when regulatory requirements demand audit trails, multiple teams coordinate across services, or AI-generated code needs human approval before merging.
- **Spec-as-source development** is the furthest end of the spectrum: specs literally become source code. The [ThoughtWorks Technology Radar \(Volume 33, 2025\)](#) places SDD in the "Assess" ring and warns of "a bias toward heavy up-front specification and big-bang releases" as an antipattern.

The Adversarial Agent Pattern



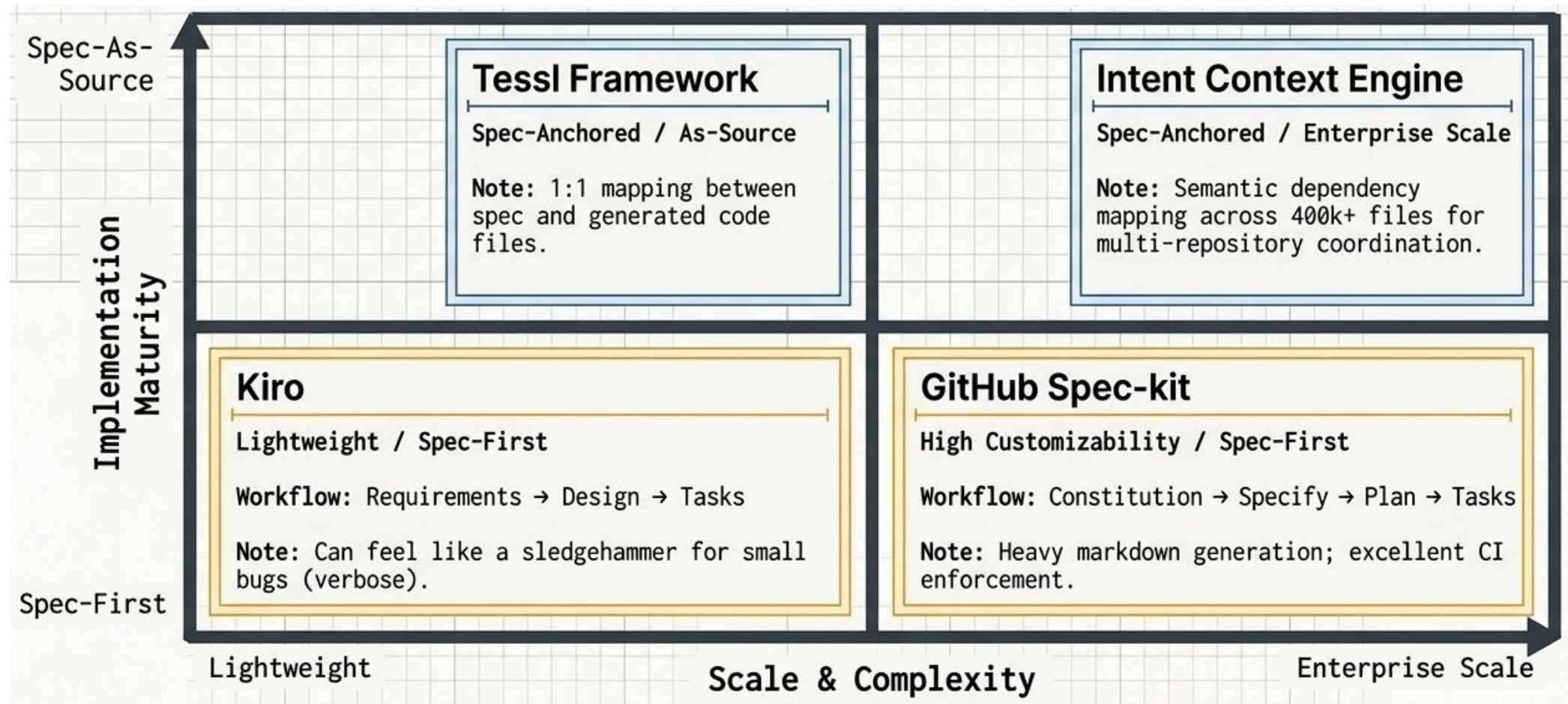
When to Use a Spec, and When Not To



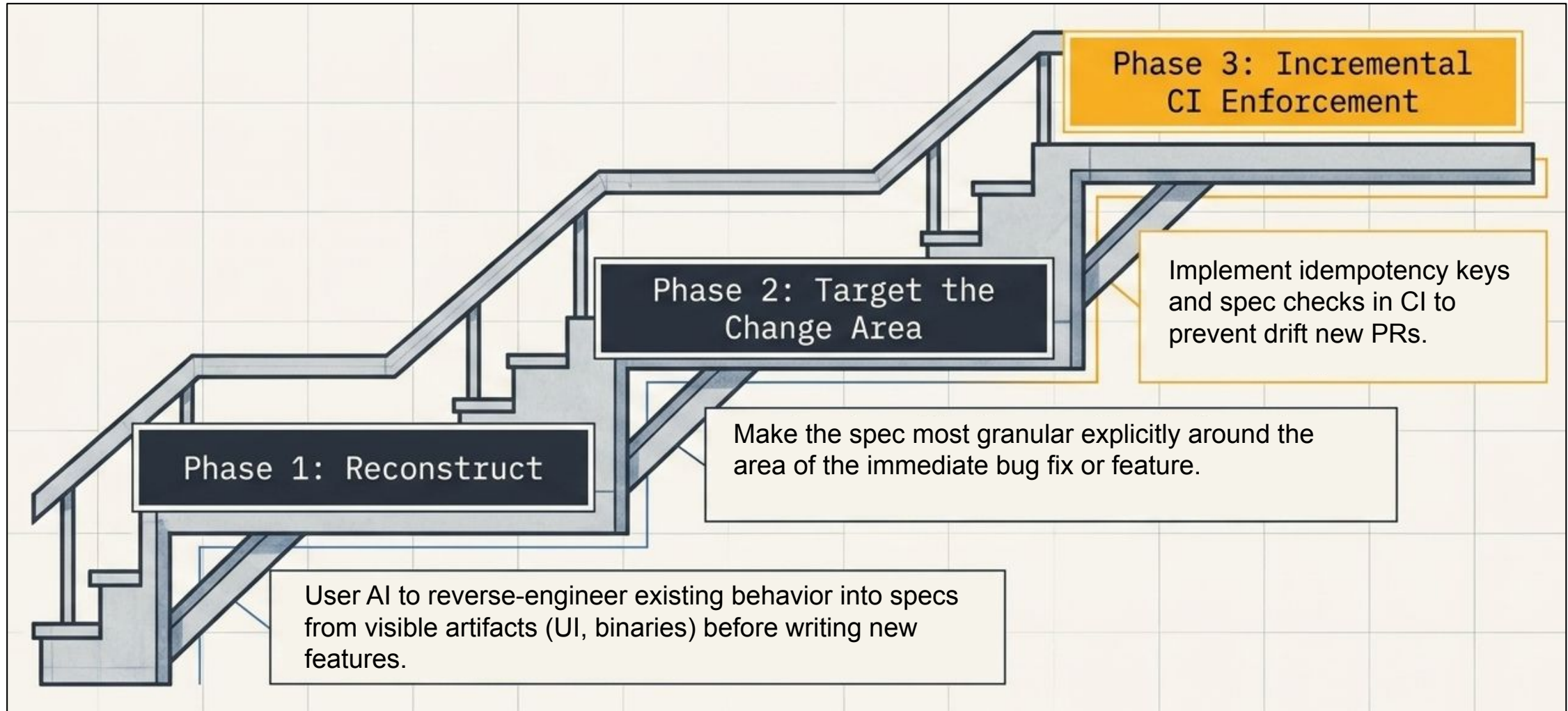
When to Use a Spec, and When Not To (cont.)

Write a Spec When	Skip the Spec When
Work spans multiple agent sessions	Work is exploratory or experimental
Multiple services or repositories are involved	A single prompt can produce usable output
Reversing a wrong interpretation is expensive	Output can be reviewed in under five minutes
Compliance or audit trail is required	Prototype is meant to be thrown away
Review will require real attention (component logic, end-to-end flows)	Change is mechanical or low-risk

Navigating the SDD Tooling Landscape



Brownfield Adoption: Applying SDD to Existing Codebases



Next week, Agentic SDLC